

IHK Abschlussprüfung Teil 2 – Masterdatei

(Deutsch + فارسی)

Diese Datei enthält alle wichtigen Algorithmen und Theorie-Themen für die IHK-Prüfung (Teil 2). Sie ist so formatiert, dass beim Druck in PDF keine Codeblöcke über zwei Seiten verteilt werden.

این فایل تمام الگوریتم‌ها و مباحث مهم تئوری امتحان IHK (بخش ۲) را شامل می‌شود و طوری تنظیم شده که در هنگام چاپ به PDF، هیچ بلوک کدی بین دو صفحه نصف نشود.

1. Sortieralgorithmen – الگوریتم‌های مرتب‌سازی

1.1 BubbleSort (Array von Integer)

Deutsch

```
prozedur sortiereBubble(a: Array von Integer)
  n = a.length
  für i = 0 bis n - 2
    für j = 0 bis n - i - 2
      wenn a[j] > a[j+1]
        temp = a[j]
        a[j] = a[j+1]
        a[j+1] = temp
      ende wenn
    ende für
  ende für
endprozedur
```

فارسی

BubbleSort همیشه عناصر کناری (همسایه‌ها) با هم مقایسه می‌شوند. اگر عنصر سمت چپ از عنصر سمت راست بزرگ‌تر باشد، جایشان عوض می‌شود. این کار در چند «پاس» تکرار می‌شود تا آرایه کاملاً صعودی مرتب شود.

1.2 BubbleSort für Objekte – مرتب‌سازی اشیاء

Deutsch

```
void sort(raeume: Array von Raum)
  n = raeume.length
  für i = 0 bis n - 2
    für j = 0 bis n - i - 2
      wenn raeume[j].getBelegung() > raeume[j+1].getBelegung()
        Raum r = raeume[j]
        raeume[j] = raeume[j+1]
        raeume[j+1] = r
      ende wenn
    ende für
  ende für
end void
```

در این نسخه، ما روی آرایه‌ای از اشیاء `Raum` کار می‌کنیم و به‌جای مقایسه مستقیم مقدار، متد `getBelegung()` را صدا می‌زنیم. منطق همان BubbleSort است، فقط مقایسه بر اساس یک ویژگی (Attribut) انجام می‌شود.

نسخه جنریک – 1.3 Generischer BubbleSort mit Function<T>

Deutsch

```
void sort(liste: List<T>, f: Function<T>)  
    n = liste.size()  
    für i = 0 bis n - 2  
        für j = 0 bis n - i - 2  
            // f(a, b) > 0 ⇒ a "größer" als b  
            wenn f(liste.get(i), liste.get(j+1)) > 0  
                T r = liste.get(i)  
                liste.set(i, liste.get(j+1))  
                liste.set(j+1, r)  
            ende wenn  
        ende für  
    ende für  
end void
```

در نسخه جنریک، تابع f مسئول مقایسه است. اگر $f(a,b)$ بزرگتر از صفر باشد یعنی a باید بعد از b قرار بگیرد و جابه‌جایی انجام می‌شود. این الگو در سال‌های اخیر در امتحان IHK بسیار پرتکرار بوده است.

2. Suchalgorithmen – الگوریتم‌های جستجو

2.1 Lineare Suche – جستجوی خطی

Deutsch

```
function sucheLinear(a: Array von Integer, x: Integer) : Integer
    für i = 0 bis a.length - 1
        wenn a[i] = x dann
            return i
        ende wenn
    ende für
    return -1
end function
```

در جستجوی خطی، از اندیس ۰ تا آخر آرایه، عنصر به عنصر بررسی می‌شود. اگر مقدار مورد نظر (x) پیدا شود، اندیس آن برگردانده می‌شود؛ در غیر این صورت -1 برگردانده می‌شود.

2.2 Binäre Suche – جستجوی دودویی

Deutsch

```
function sucheBinaer(a: Array von Integer, x: Integer) : Integer
    links = 0
    rechts = a.length - 1

    while links <= rechts
        mitte = (links + rechts) / 2

        wenn a[mitte] = x dann
            return mitte
        ende wenn

        wenn x < a[mitte] dann
            rechts = mitte - 1
        sonst
            links = mitte + 1
        ende wenn
    end while

    return -1
end function
```

در جستجوی دودویی، آرایه باید از قبل مرتب شده باشد. در هر مرحله عنصر وسط محاسبه می‌شود؛ اگر برابر با x باشد، جواب پیدا شده است؛ اگر x کوچک‌تر باشد، جستجو در نیمهٔ چپ ادامه می‌یابد و اگر بزرگ‌تر باشد، در نیمهٔ راست ادامه می‌دهیم.

3. Prüfziffer – رقم کنترلی (mod 10)

Deutsch

```
function ermittlePruefziffer(code: Array von Integer) : Integer
    sum = 0
    für i = 0 bis code.length - 1
        wenn i % 2 = 0 dann
            faktor = 1
        sonst
            faktor = 3
        sum = sum + code[i] * faktor
    ende für

    rest = sum % 10
    wenn rest = 0 dann
        return 0
    sonst
        return 10 - rest
    ende wenn
end function
```

در این الگوریتم، ارقام با وزنهای ۱ و ۳ به صورت یکی در میان ضرب می‌شوند و سپس، جمع کلی حساب می‌شود. بعد، باقیمانده تقسیم بر ۱۰ گرفته شده و رقم کنترلی با فرمول $10 - \text{باقیمانده}$ به دست می‌آید (اگر باقیمانده صفر بود، خود رقم کنترلی ۰ است).

4. Rekursion – بازگشت

4.1 Fakultät – فاکتوریل

Deutsch

```
function fak(n: Integer): Integer
  wenn n = 1 dann
    return 1
  sonst
    return n * fak(n - 1)
  ende wenn
end function
```

در بازگشت فاکتوریل، شرط توقف $n = 1$ است. در غیر این صورت، تابع خودش را با $n-1$ فراخوانی کرده و در برگشت، مقادیر در هم ضرب می‌شوند.

4.2 Dezimal → Binär – تبدیل دسیمال به باینری (بازگشتی)

Deutsch

```
function dezimalZuBinaer(n: Integer) : String
  wenn n < 2 dann
    return string(n)
  sonst
    return dezimalZuBinaer(n / 2) + string(n % 2)
  ende wenn
end function
```

در این روش، عدد هر بار بر ۲ تقسیم شده و تابع دوباره صدا زده می‌شود. در برگشت بازگشت (Rekursion)، باقی‌مانده‌ها به آخر رشته اضافه می‌شوند و در نهایت نمایش باینری عدد به دست می‌آید.

5. Stack & Heap – پشته و هیپ

Deutsch

- **Stack:** lokale Variablen, Parameter, Rücksprungadressen, LIFO, sehr schnell, Fehler: StackOverflow.
- **Heap:** Objekte, Arrays, Listen, vom Garbage Collector verwaltet, Fehler: OutOfMemoryError.

Stack: برای متغیرهای محلی، پارامترها و آدرس بازگشت توابع استفاده می‌شود. ساختار آن LIFO است و سرعت بالایی دارد. اگر بازگشت یا توابع بی‌نهایت فراخوانی شوند، خطای `StackOverflow` رخ می‌دهد.

Heap: برای ذخیره اشیاء، آرایه‌ها و ساختارهای داده‌ای یوما استفاده می‌شود و توسط Garbage Collector مدیریت می‌شود. اگر حافظه پر شود، خطای `OutOfMemoryError` ایجاد می‌شود.

6. Generics – جنریک‌ها

Deutsch

- `List<T>` : generische Liste, speichert Elemente eines Typs T.
- `Function<T>` : Vergleichsfunktion, z. B. `f(a, b)` mit Rückgabewert `> 0`, `< 0`, `= 0`.
- T ist ein Typparameter, der zur Compile-Zeit ersetzt wird.

در جنریک‌ها، T به عنوان نوع پارامتر استفاده می‌شود. `<List<T` می‌تواند برای هر نوع (مثلاً، `Integer`، `Raum`، `Artikel`) استفاده شود. معمولاً برای مقایسه دو شیء به کار می‌رود؛ اگر خروجی آن `<` 0 باشد یعنی شیء اول «بزرگ‌تر» است.

7. Theoriefragen – سوالات تئوری

Deutsch – typische Fragen

1. Erläutern Sie den Unterschied zwischen Stack und Heap.
2. Was ist eine generische Klasse?
3. Nennen Sie einen Vorteil der Rekursion.
4. Welche Aufgaben hat ein Konstruktor?
5. Erläutern Sie den Unterschied zwischen Klasse und Objekt.
6. Warum ist BubbleSort ineffizient?
7. Was bedeutet $O(n^2)$?

فارسی – معادل سؤالها

(۱) تفاوت Stack و Heap را توضیح دهید. (۲) کلاس جنریک چیست؟ (۳) یکی از مزایای بازگشت را نام ببرید. (۴) وظیفه سازنده (Konstruktor) چیست؟ (۵) تفاوت کلاس و شیء چیست؟ (۶) چرا BubbleSort از نظر کارایی ضعیف است؟ (۷) منظور از $O(n^2)$ چیست؟

8. Häufige Fehler – اشتباهات رایج

- `j+1` statt `j+1` (Tippfehler in Schleifen).
- Vergessen von `n = liste.size()` vor Schleifen.
- Falsche Schleifengrenzen (Off-by-one-Fehler).
- Kein `return` in rekursiven Funktionen.
- Vergleich von Objekten ohne Getter.

اشتباهات کلاسیک در امتحان:

- نوشتن `j+1` به جای `j+1`.
 - فراموش کردن تعریف `n = liste.size()` قبل از حلقه‌ها.
 - مرزهای اشتباه در حلقه‌ها (Off-by-one).
 - نداشتن `return` در توابع بازگشتی.
 - مقایسهٔ اشیاء بدون استفاده از Getter.
-

9. Beispiele aus IHK-Prüfungen (2015–2024) – مثال‌های امتحانی

2015 – Minimum in Array

```
function findeMinimum(a)
  min = a[0]
  für i = 1 bis a.length - 1
    wenn a[i] < min dann
      min = a[i]
    ende wenn
  ende für
  return min
end function
```

2016 – Durchschnitt berechnen

```
function durchschnitt(a)
  sum = 0
  für i = 0 bis a.length - 1
    sum = sum + a[i]
  ende für
  return sum / a.length
end function
```

2019 – Lineare Suche (Objekte)

```
function suche(liste, x)
  für i = 0 bis liste.size() - 1
    wenn liste.get(i).getId() = x dann
      return i
    ende wenn
  ende für
  return -1
end function
```

2020 – Passwortprüfung (mindestens 3 Ziffern)

```

function pruefe(pw)
    count = 0
    für i = 0 bis pw.length - 1
        wenn istZiffer(pw[i]) dann
            count = count + 1
        ende wenn
    ende für
    wenn count >= 3 dann
        return true
    sonst
        return false
    ende wenn
end function

```

2021 – Rekursion (Zahlen rückwärts)

```

function ruckwaerts(n)
    wenn n = 0 dann
        return
    ende wenn
    print(n)
    ruckwaerts(n - 1)
end function

```

2024 – Zeichen zählen

```

function countChar(text, c)
    count = 0
    für i = 0 bis text.length - 1
        wenn text[i] = c dann
            count = count + 1
        ende wenn
    ende für
    return count
end function

```

10. Prognose-Beispiele 2025 – مثال‌های پیش‌بینی‌شده ۲۰۲۵

10.1 sortArtikel + Prüfwert

```
void sortArtikel(liste)
  n = liste.size()
  für i = 0 bis n - 2
    für j = 0 bis n - i - 2
      wenn liste.get(j).getPreis() > liste.get(j+1).getPreis()
        t = liste.get(j)
        liste.set(j, liste.get(j+1))
        liste.set(j+1, t)
      ende wenn
    ende für
  ende für
end void
```

10.2 Lineare Suche + rekursive Ausgabe

```
function suche(liste, x)
  für i = 0 bis liste.size() - 1
    wenn liste.get(i).equals(x) dann
      return i
    ende wenn
  ende für
  return -1
end function
```

```
function ausgabeRekursiv(liste, index)
  wenn index = liste.size() dann
    return
  ende wenn
  print(liste.get(index))
  ausgabeRekursiv(liste, index + 1)
end function
```

10.3 Passwortobjekte prüfen

```
function pruefe(pw)
    gross = 0
    digits = 0
    für i = 0 bis pw.length - 1
        wenn istGross(pw[i]) dann
            gross = gross + 1
        ende wenn
        wenn istZiffer(pw[i]) dann
            digits = digits + 1
        ende wenn
    ende für
    wenn gross >= 2 UND digits >= 3 UND pw.length > 8 dann
        return true
    sonst
        return false
    ende wenn
end function
```

11. Activity Diagram – توضیحات دیاگرام فعالیت

11.1 BubbleSort – Beschreibung

Start → $i = 0$ → Entscheidung ($i < n-1?$) → j-Schleife ($j < n-i-1?$) → Vergleich $a[j]$ und $a[j+1]$ → ggf. Tausch → $j++$ → zurück zur j-Schleife → $i++$ → zurück zur i-Schleife → Ende.

در Activity Diagram برای BubbleSort، دو حلقه تو در تو با گره‌های تصمیم رسم می‌شود؛ درون حلقه j یک شرط مقایسه و در صورت نیاز عمل Swap انجام می‌شود، سپس به ابتدای حلقه برمی‌گردیم تا آرایه مرتب شود.

11.2 Lineare Suche – Beschreibung

Start → $i = 0$ → Entscheidung ($i < n?$) → Vergleich $a[i]$ mit x → wenn gleich: Ergebnis = i , Ende → sonst $i++$ und zurück.

11.3 Binäre Suche – Beschreibung

Start → links = 0, rechts = $n-1$ → Entscheidung ($\text{links} \leq \text{rechts}?$) → mitte berechnen → Vergleich $a[\text{mitte}]$ mit x → falls gleich: Ergebnis, Ende → sonst Entscheidung $x < a[\text{mitte}]?$ → rechts oder links anpassen → zurück.

11.4 Prüfwert – Beschreibung

Start → sum = 0, $i = 0$ → Schleife über alle Ziffern → Gewicht bestimmen (1 oder 3) → sum aktualisieren → $i++$ → nach der Schleife: $\text{rest} = \text{sum} \% 10$ → Prüfwert bestimmen → Ende.

12. Merksätze – جملات حفظی

12.1 BubbleSort

„Nachbarn vergleichen, bis nichts mehr zu tauschen ist.“

«همسایه‌ها را مقایسه کن تا دیگر هیچ جابه‌جایی لازم نباشد.»

12.2 Lineare Suche

„Von vorne bis hinten jedes Element prüfen.“

«از اول تا آخر، تک‌تک عناصر را بررسی کن.»

12.3 Binäre Suche

„Immer wieder die Mitte suchen – links oder rechts weiter.“

«هر بار وسط را پیدا کن؛ یا به چپ برو یا به راست.»

12.4 Prüfstoff

„Gewichte anwenden, Summe bilden, Modulo, 10 minus Rest.“

«وزن‌ها را اعمال کن، جمع بگیر، باقیمانده ۱۰ را حساب کن، ۱۰ منهای باقی‌مانده رقم کنترلی است.»

Ende der Masterdatei – PDF-optimierte Version.

پایان فایل مستر – نسخه بهینه برای PDF.